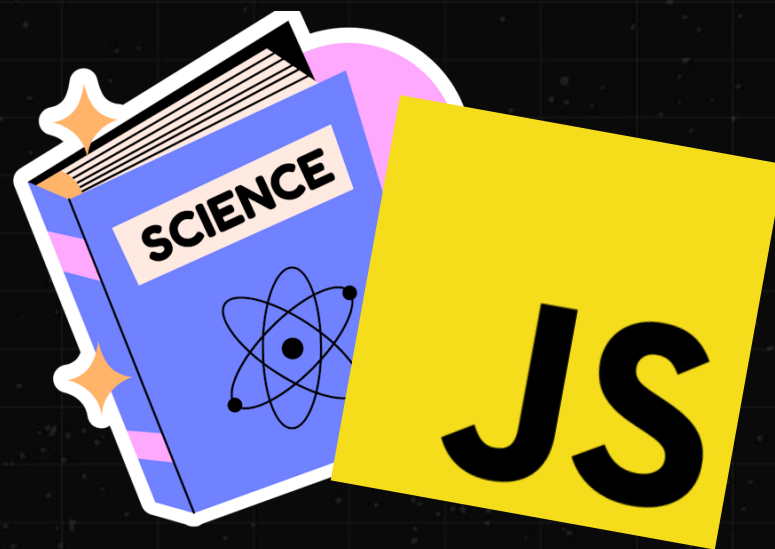


ЛЕКЦИЯ #18

CS BO FRONTEND



ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Поговорить про **способы кодирования строк**
- * Как это делали раньше и **как это устроено сейчас**
- * Реализация строк в JavaScript
- * Нюансы работы с юникодом

СТРОКА СОСТОИТ ИЗ СИМВОЛОВ

- * Символ — это **атомарная конструкция**
- * Символ может представлять любую цифру или букву любого алфавита
- * А также смайлики, флажки и так далее
- * **Пробел или перевод строки** — это тоже символы
- * Значит, строка — это просто **последовательный поток символов**
- * Раз в строке не может быть «дырок», то для её представления **удобно использовать массив или вектор**

ЕСЛИ СТРОКА ЭТО
МАССИВ, ТО ВСЕ
СИМВОЛЫ ДОЛЖНЫ
ИМЕТЬ **ОДИНАКОВЫЙ**
РАЗМЕР...



ВСЕ ТАК, ЕСЛИ СТРОКА – ЭТО МАССИВ

- * Но какой тогда должен быть **размер одного символа?**
- * В 8 битах можно закодировать **256 различных комбинаций**
- * Это обеспечивает **быстрый доступ по индексу** к любому символу в строке
- * Можно каждой комбинации бит поставить в соответствие отдельный символ
- * Таблица, определяющая, какой комбинации бит соответствует какой символ, называется **кодовой страницей** (кодовой таблицей)

256 СИМВОЛОВ СЛИШКОМ МАЛО

- * Но использовать много разных алфавитов в одной строке нужно не так часто
- * Поэтому можно ввести множество разных кодовых страниц и использовать нужную в зависимости от ситуации
- * Например, Latin-1, CP-1251, KOI-8
- * Однако теперь нужно знать, **какую кодовую страницу использовать** для открытия документа

А ЧТО ЕСЛИ ПРОЧИТАТЬ
СТРОКУ ЧЕРЕЗ ДРУГУЮ
КОДОВУЮ СТРАНИЦУ?



БУДУТ ПРОБЛЕМЫ

၁'N< ၁%၄၃%၄၇၉μN,၅μ ၁'½၀°N□N,ÑE၁%၄_Ñ,NÆ N□၀²D¾ÑZ ၀°၀»၀°၀²D_၀°N,ÑfÑ€Ñf
၀´၀»N□ Ñ€ÑfN□N□၀°၀%၄¹ ÑE၀°N□၀°၀»၀°၀´၀°၀_ Windows XP - N□N,၀°၀´½၀´၀°
Ñ€Ñ,၀´½၀°N□ Ñ€ÑfN□N□၀°၀°N□ ၀°၀»၀°၀²D_၀°N,ÑfÑ€၀°

Đ'Đ°Đ¶Đ½Đ¾! Đ§Ń,Đ¾Đ±Ń< Đ´Đ¾Đ±Đ°Đ²Đ, Ń,ŃĐĐ ŃĐĐŃ□Đ°Đ»Đ°Đ ´Đ°Ńđ Đ°Đ»Đ°Đ
Đ²Đ,Đ°Ń,ŃđŃŃŃ< Đ¿Đ¾Đ´ Windows XP/2003, Đ¿Đ¾Đ»ŃĐĐ·Đ¾Đ²Đ°Ń,ĐµĐ»ŃĐĐ
Đ´Đ¾Đ»Đ¶ĐµĐ½ Đ²Đ¾Đ¹Ń,Đ² Ń□Đ, Ń□Ń,ĐµĐ¼Ńđ Đ°Đ°Đ°
Đ□Đ´Đ¼Đ,Đ½Đ,Ń□Ń,ŃĐĐŃ,Đ¾ŃŃ,Đ,Đ½Đ°Ń≠Đµ Đ² Ń□Đ»ĐµĐ´ŃđŃŽŃ‰ĐµĐ¹
Ń□ĐµŃŃ□Đ,Đ, Ń□Ń,Đ... ŃđŃ□Ń,Đ°Đ½Đ¾Đ²Đ¾Đ°Đ½Đµ Đ±ŃđĐ´ĐµŃ, -Đ°Đ°Đ°
Đ,Đ½Đµ ŃđŃ□Ń,Đ°Đ½Đ°Đ²Đ»Đ,Đ²Đ°Đ»Đ!

ĐŠĐ°Đ° Đ°Đ°Ń,Đ_Đ²Đ_Ń€Đ¼Đ²Đ°Ń,Ń€ Ń□Ń,Đ°Đ°½Đ' Đ°Ń€Ń,Đ°½Ń_fŃŽ Ń€Đ°
Ń□Đ°Đ°»Đ°Đ' Đ°Ń_f Ń€Ń_fŃ□Ń□Đ°Đ°¼Đ± Đ°Đ°»Đ°Đ²Đ_Đ°Ń,Ń_fŃ€Ń< Đ_Đ¼Đ¼' XP / 2003:

РАЗВИТИЕ IT ЗАРОЖДАЛОСЬ НА ЗАПАДЕ

- * Поэтому многие решения появлялись для решения задач, **актуальных для конкретного региона**
- * Так, первые кодовые таблицы включали в себя латинский алфавит (заглавные и строчные), цифры, знак доллара и другое
- * Но и здесь не было универсального решения – получился зоопарк кодировок
- * В конечном счёте стало понятно, что **нужна стандартизация**

ТАК ПОЯВИЛСЯ ASCII

- * Стандартная кодовая страница для латинского алфавита
- * Размер одного символа – 7 бит
- * **Всего 128 символов**
- * Так как размер байта равен 8 битам, **один бит остался свободным**
- * Его можно использовать для **кодирования ещё 128 символов**
- * Так и появились **ASCII-совместимые кодовые страницы**
- * Например, Windows-1251 (CP-1251) для кириллицы

Кодировка ASCII WINDOWS CP1251

Цифры
0..9
48..57

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0				©	€	§	€	.		°						
1		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
2			!	"	#	\$	%	&	(	)		+				/
3	0	1	2	3	4	5	6	7	8	9			;	<	=	>
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	(	\		^	_
6	'	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	□
8	б	г	д	е	ж	з	и	й	к	л	м	н	о	п	р	с
9	т	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я			
A	у	ѳ	ј	џ	ѧ	Ѩ	ѩ	Ѫ	ѫ	Ѭ	ѭ	Ѯ	ѯ	Ѱ	ѱ	Ѳ
B	ѳ	ѧ	Ѩ	ѩ	Ѫ	ѫ	Ѭ	ѭ	Ѯ	ѯ	Ѱ	ѱ	Ѳ	ѳ	Ѵ	ѵ
C	Ѷ	ѷ	Ѹ	ѹ	Ѻ	ѻ	Ѽ	ѽ	Ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ
D	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ
E	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ
F	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ	ѿ

Латиница
A..Z 65..90
a..z 97..122

Кирилица
А..Я
192..223
а..я
224..255

LATIN-1 (ISO-8859-1)

- ✱ Ещё одна ASCII-совместимая кодовая страница
- ✱ Добавляет буквы западноевропейских алфавитов: á, é, ñ, ü, ß, ç, ø, æ и другие
- ✱ А также специальные и технические символы: ©, ®, °, ±, ¼, ½, ¿, «, » и другие
- ✱ Фактически, это расширение ASCII для стран Западной Европы

НО ЕСЛИ НУЖНО
СМЕШИВАТЬ РАЗНЫЕ
АЛФАВИТЫ ВМЕСТЕ?



А ВОТ С ЭТИМ ПРОБЛЕМА

- ✱ Как вариант, можно использовать «костыль» и ввести специальные последовательности символов, которые **отображаются как новые символы**
- ✱ Например, [HTML-мнемоники](#)
- ✱ Однако это **усложняет парсинг и рендеринг такой строки**
- ✱ А также **ломает произвольный доступ** по индексу строки

”	„	U+201E	Нижняя двойная кавычка
’	& #10076;	U+275C	Жирная одиночная верхняя запятая
‘	& #10075;	U+275B	Жирная одиночная повёрнутая верхняя запятая
&	&	U+0026	Амперсанд
'	'	U+0027	Апостроф (одинарная кавычка)
§	§	U+00A7	Параграф
©	©	U+00A9	Знак copyright
¬	¬	U+00AC	Знак отрицания
®	®	U+00AE	Знак зарегистрированной торговой марки
—	¯	U+00AF	Знак долготы над гласным
°	°	U+00B0	Градус
±	±	U+00B1	Плюс-минус
1	¹	U+00B9	Верхний индекс "1"
2	²	U+00B2	Верхний индекс "2"
3	³	U+00B3	Верхний индекс "3"
¼	¼	U+00BC	Одна четверть

А ЕСЛИ
КОДИРОВАТЬ
В 2 БАЙТА?



UCS-2

- * Новая кодовая страница, **не совместимая с ASCII**
- * Каждый символ **кодируется двумя байтами** (UCS-2)
- * Может вмещать более 65 тысяч символов
- * Успех?

ЕСТЬ НЮАНСЫ

- * Один символ занимает 2 байта
- * Потребление памяти на строки **выросло в 2 раза** по сравнению с ASCII
- * При передаче информации между компьютерами появляется новый нюанс — **порядок следования байтов**, который указывается отдельным флагом (BOM)
- * **65 тысяч символов недостаточно**, чтобы закодировать все алфавиты мира

ТОГДА НУЖНО
ИСПОЛЬЗОВАТЬ
4 БАЙТА?



ТАК МОЖНО

- * Хотя сейчас по факту **нам хватило бы 21 бита**, но это неудобно использовать
- * Но потребление памяти **увеличивается в 4 раза**
- * Даже если мы **не используем разные алфавиты**
- * **Теряется обратная совместимость с ASCII**
- * Нужен способ кодировать новые символы, **не увеличивая размер одного символа**

В UCS-2 БЫЛ НЕКОТОРЫЙ «ЗАПАС»

- * Отведённых под будущие нужды
- * Взяли 1024 **символа** и **ввели специальные символы** (code units) – старшие суррогаты (high surrogates)
- * Взяли ещё 1024 code units и **ввели младшие суррогаты** (low surrogates)
- * Комбинация старшего и младшего суррогата позволяет закодировать **более миллиона новых символов**
- * Вместо простой кодовой страницы появляется новый подход: **набор символов (Юникод) и кодировка (UTF-16)**

РЕФЛЕКСИРУЕМ

- * Запас по символам теперь большой
- * В большинстве случаев **хватает 16 бит на символ**
- * Но некоторые символы (например, смайлики или некоторые иероглифы) **кодируются двумя 16-битными code units – суррогатной парой**
- * Усложняется парсинг строк и появляется понятие «кодировка»
- * Если в строке есть суррогаты, то **произвольный доступ по индексу** становится невозможным (или требует специальной обработки)

ЧТО ЕЩЕ ЗА
ЮНИКОД?



ЮНИКОД

- * Раньше кодовая страница определяла и **набор символов, и способ их кодирования**
- * В Юникоде же эти понятия разделены: есть набор символов, где каждый символ имеет **свою кодовую позицию** (code point) – 32 бита (по факту 21 – остальное 0)
- * А также есть алгоритмы кодирования, которые **упаковывают эти коды** в меньшее количество байт – кодировки (UTF-8, UTF-16, UTF-32)

UTF-16

- * Расширение UCS-2
- * Память: **в 2 раза больше, чем ASCII** и нет совместимости
- * **Random access есть, но сломан** (суррогатные пары)
- * **Проблема порядка байт (BOM)**
- * **Очень распространён в различных языках программирования** (Java, JavaScript, C#, Windows API)

UTF-32

- * Каждый символ кодируется как его Code Point (32 бита)
- * Random access есть (на самом деле он тоже сломан, но по другим причинам)
- * **Порядок байт (BOM) так же актуален**, как и в UTF-16
- * **Память: в 4 раза больше, чем ASCII** (даже для латиницы)

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ИНТЕРНЕТ ИЗМЕНИЛ ВСЕ

- * С развитием интернета стало понятно, что размер **файлов снова очень важен**
- * Кодирование с **фиксированным размером (x2, x4)** часто избыточно
- * А использовать **разные кодовые страницы – геморройно**
(поддерживать множество кодировок, угадывать их, конвертировать)
- * Константный **доступ по номеру символа в большинстве сценариев не нужен**
- * Нужна **кодировка с переменной длиной символа** – экономичная и совместимая с ASCII

UTF-8

- * Совместима с ASCII
- * Каждый символ кодируется от 1 до 4 байт (в зависимости от диапазона)
- * Нет безопасного произвольного доступа по индексу (переменная длина)
- * Нет проблем с порядком байт (побайтовая кодировка)
- * Очень эффективный расход памяти (латиница – 1 байт)
- * Де-факто стандарт в вебе, API, файлах и повсеместно

Предложение

Условные обозначения

1 байт (ASCII)

1-й байт (2 байта)

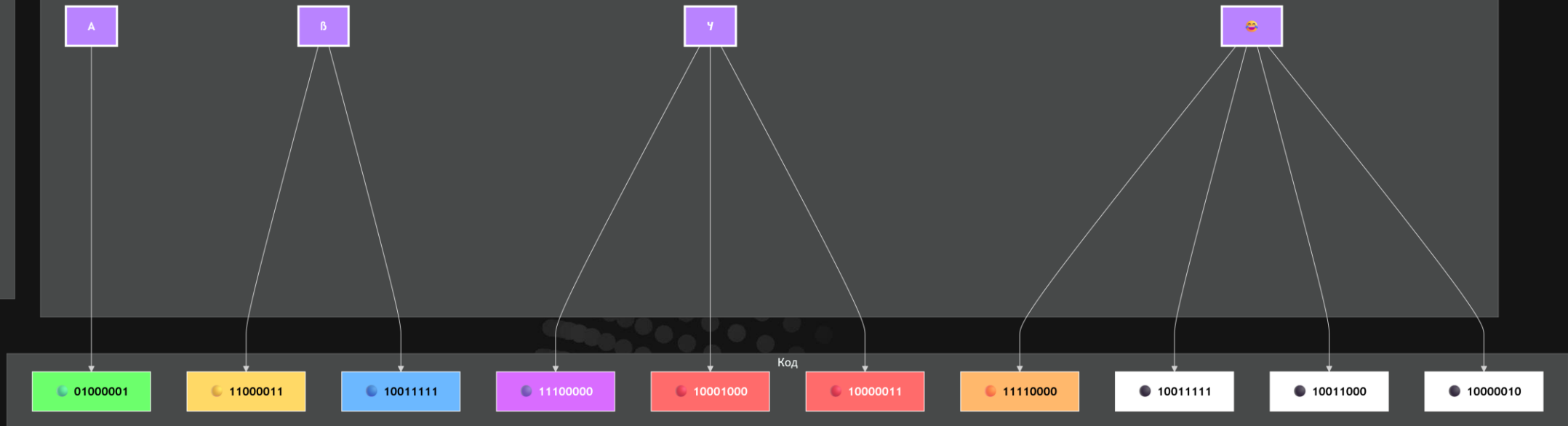
2-й байт

1-й байт (3 байта)

2-й, 3-й байты

1-й байт (4 байта)

2-й, 3-й, 4-й байты



РЕФЛЕКСИРУЕМ

- ✱ UTF-16 — компромиссная кодировка, которая **часто оказывается неудобной**
- ✱ UTF-32 в **большинстве сценариев избыточна**, но можно использовать для символов
- ✱ UTF-8 в **большинстве случаев — лучший выбор**
(латиница — 1 байт, совместимость с ASCII, нет BOM)

НО В JS ЖЕ
UTF-16?



ДА

```
// Кодировка через суррогатную пару
console.assert("🔥".length === 2);

// Старший суррогат
console.log("🔥".charAt(0) === "\ud83d");

// Младший суррогат
console.log("🔥".charAt(1) === "\udd25");

// Code-point в Unicode
console.log("🔥".codePointAt(0) === 128293);
```

ИСПОЛЬЗУЙТЕ ИТЕРАТОРЫ

```

// ["\ud83d", "\udd25"]
console.log("🔥".split(""));

// ["🔥"]
console.log([... "🔥"]);

for (const char of "🔥🔥🔥") {
  console.log(char);
}
```


ИСПОЛЬЗУЙТЕ ФЛАГ u

```
console.assert(/[😄-😏]/u.test("😄") === true);

// Uncaught SyntaxError: invalid range in character class
/[😄-😏]/.test("😄");
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

НУЖНО ПОНИМАТЬ

- * Даже если текстовые файлы (HTML, CSS, JS) вы сохраняете в UTF-8, то **JS VM хранит строки как последовательности 16-битных code units**
- * Так исторически сложилось и **того требует спецификация языка**
- * Технически, строка **может содержать непарные суррогаты**, что неверно относительно UTF-16
- * В современных языках программирования внутреннее представление строк – как правило UTF-8 (Go, Rust)

ЕЩЕ ПРИМЕР

- * Для передачи текстовой информации по сети стандартом де-факто является UTF-8
- * Однако в JS нет встроенного строкового типа для этой кодировки – строки всегда в UTF-16
- * Переконвертация в UTF-16 может быть избыточной, поэтому в Node.js данные по умолчанию приходят как Buffer (бинарные данные)
- * Конвертация в строку (UTF-16) происходит только по запросу
- * Современные API (fetch, Streams) следуют тому же принципу


```
// ===== TextEncoder: строка → Uint8Array (UTF-8) =====

const encoder = new TextEncoder();

const str = "Hello, Привет 🍕 ";
const uint8Array = encoder.encode(str);

console.log("Исходная строка:", str);
console.log("Длина строки (UTF-16 символы):", str.length);
console.log("Длина Uint8Array (байты в UTF-8):", uint8Array.length);
console.log("Байты:", uint8Array);

// ===== TextDecoder: Uint8Array → строка =====

const decoder = new TextDecoder();

const decodedStr = decoder.decode(uint8Array);
console.log("Декодированная строка:", decodedStr);
console.log("Строки совпадают:", str === decodedStr);
```

```
// ===== Демонстрация разницы в размере =====

const englishText = "Hello World";
const russianText = "Привет Мир";
const emojiText = "🍕 🤖 🚀";

console.log("\n=== Размер в памяти ===");

// "Hello World" | UTF-16: 22 байт | UTF-8: 11 байт
console.log(`"${englishText}" | UTF-16: ${englishText.length * 2} байт | UTF-8: ${encoder.encode(englishText).length} байт`);

// "Привет Мир" | UTF-16: 20 байт | UTF-8: 19 байт
console.log(`"${russianText}" | UTF-16: ${russianText.length * 2} байт | UTF-8: ${encoder.encode(russianText).length} байт`);

// "🍕 🤖 🚀" | UTF-16: 12 байт | UTF-8: 12 байт/
console.log(`"${emojiText}" | UTF-16: ${emojiText.length * 2} байт | UTF-8: ${encoder.encode(emojiText).length} байт`);
```

ЕЩЕ НЮАНСЫ СО СТРОКАМИ В JS

- * Строки в JS являются неизменяемыми примитивами и, зачастую, **представляются как указатель на значение в куче**
- * Любая операция изменения строки **создаёт новую строку** (исходная остаётся неизменной)
- * Однако современные движки **оптимизируют конкатенацию**: вместо немедленного копирования может создаваться специальный объект
- * Этот объект просто **хранит указатели на фрагменты строк**
- * Фактическое склеивание произойдёт только при необходимости



```
// Используем repeat, чтобы движок не смог сразу применить оптимизацию и объединить строки
let a = "Hello".repeat(2);
let b = "World".repeat(2);

let c = a + b;

%DebugPrint(c); // CONS_ONE_BYTE_STRING_TYPE

// Провоцируем объединение
%DebugPrint(c.split("").join("")); // SEQ_ONE_BYTE_STRING_TYPE
```

ОБРАТИТЕ ВНИМАНИЕ

- * V8 создаёт отдельные типы для однобайтовых (Latin-1) строк – такие строки **не содержат символов, выходящих за пределы 1 байта**
- * В результате для такой строки **будет использоваться в 2 раза меньше памяти**
- * Помимо ConsString и SeqString, в V8 есть и другие подвиды строк



```
let a = "Hello мир";  
let b = "Привет World".repeat(2);  
  
let c = a + b;  
  
%DebugPrint(c); // CONS_TWO_BYTE_STRING_TYPE  
  
// Провоцируем объединение  
%DebugPrint(c.split("").join("")); // SEQ_TWO_BYTE_STRING_TYPE
```



```
let a = "Hello мир";
```

```
let b = "Hello мир";
```

```
// Два одинаковых литерала - это ссылка на одну и ту же строку
```

```
// V8 интернирует строковые литералы (interned strings)
```

```
// DebugPrint: 000000618B821899: [String] in OldSpace: u#Hello \u043c\u0438\u0440
```

```
// 000003F56B580201: [Map] in ReadOnlySpace
```

```
// - type: INTERNALIZED_TWO_BYTE_STRING_TYPE
```

```
%DebugPrint(a);
```

```
// DebugPrint: 000000618B821899: [String] in OldSpace: u#Hello \u043c\u0438\u0440
```

```
// 000003F56B580201: [Map] in ReadOnlySpace
```

```
// - type: INTERNALIZED_TWO_BYTE_STRING_TYPE
```

```
%DebugPrint(b);
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЖКА, СТОЖКААА!

ВЫДЫХАЕМ

- * В JS из коробки **нет встроенного типа для изменяемых строк** — строки неизменяемы
- * Поэтому виртуальной машине **приходится идти на различные оптимизации** (ConsString, SlicedString, интернирование)
- * Можно сказать, что **интенсивная работа со строками — не самая сильная сторона JS** (особенно в старых движках)
- * Поэтому **некоторые парсеры, транспиляторы и инструменты для текста** могут быть куда эффективнее реализованы на других языках (Rust, Go, C++)

ЕЩЕ НЮАНСЫ ПРО ЮНИКОД

- * В Unicode есть **диакритические символы** (комбинируемые), которые добавляются к буквам
- * Например, буква И + диакритический знак ǐ (комбинирующая кратка) → Й
- * Для обратной совместимости в Unicode также **существует отдельный символ Й**
- * **С эмодзи ситуация ещё сложнее:** у них можно менять цвет кожи (модификаторы) и комбинировать несколько эмодзи в одно
- * Одна графема (пользовательский символ) **может состоять из нескольких Unicode-кодов** (sequence)

НАПРИМЕР

```
console.assert("💖".length === 13);

// [
//   '💖', ' ',
//   '💖', ' ',
//   '💖', ' ', '💖'
// ]
console.log([... "💖💖"]);
```

ОДНУ И ТУ ЖЕ СТРОКУ МОЖНО ПРЕДСТАВИТЬ ПО-РАЗНОМУ В ЮНИКОДЕ



```
console.assert("й".length === 2);  
console.assert("й".length === 1);  
console.assert("й" !== "й");
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ЕСЛИ ВЫ ИСПУГАЛИСЬ – ЭТО ХОРОШО

- * Действительно, используя такие особенности Unicode, можно придумать **хитрые способы подделки одного контента под другой**
- * Например, создать клон страницы в соцсети **с визуально идентичным**, но технически разным именем пользователя
- * Чтобы бороться с такими проблемами, в Unicode были введены **нормальные формы**

НАПРИМЕР



```
console.assert("й".length === 2);  
console.assert("й".length === 1);  
  
console.assert("й" !== "й");  
  
console.assert("й".normalize().length === 1);  
console.assert("й".normalize("NFD").length === 2);  
console.assert("й".normalize() === "й".normalize());
```

НОРМАЛИЗОВАТЬ МОЖНО ПО-РАЗНОМУ

- * **NFC** – приводит строку к каноническому виду (собирает диакритику)
- * **NFD** – разлагает символы на составные части
- * **NFKC / NFKD** – более агрессивные (совместимые) формы
- * По умолчанию используется **NFC**

ВСЕГДА НОРМАЛИЗУЙТЕ СТРОКИ

- * Когда **принимаете их от третьей стороны** (пользовательский ввод, API, файлы)
- * Визуально одна и та же строка может давать **разный хэш-код**
- * Это может использоваться для мошенничества, обхода проверок, дублирования ключей

ИСПОЛЬЗУЙТЕ МЕТОДЫ СТРОКИ



```
console.assert("й".length === 2);  
console.assert("й".length === 1);  
  
console.assert("й" !== "Й");  
console.assert("й".localeCompare("Й") === 0);
```

ПОМНИТЕ ПРО ЛОКАЛЬ



```
console.assert("i".toUpperCase() === "I");  
console.assert("i".toLocaleUpperCase("tr") === "İ");
```

ПРАВИЛЬНОЕ СРАВНЕНИЕ СТРОК



```
// В русском алфавите "ё" - отдельная буква, но сравнение ставит её рядом с "е"
const russian = ["елка", "ёлка", "еж", "ёж"];

const collator = new Intl.Collator("ru");

console.log(russian.toSorted());           // [ 'еж', 'елка', 'ёж', 'ёлка' ]
console.log(russian.toSorted(collator.compare)); // [ 'еж', 'ёж', 'елка', 'ёлка' ]
```

```
// 1. Создаём сегментатор для графем (пользовательских символов)
const graphemeSplitter = new Intl.Segmenter("ru", { granularity: "grapheme" });

// { segment: '💖', index: 0, input: '💖👉 é' }
// { segment: 'é', index: 13, input: '💖👉 é' }
console.log(...graphemeSplitter.segment("💖👉 é"));

// 2. Сегментация на слова
const wordSplitter = new Intl.Segmenter("ru", { granularity: "word" });

// { segment: 'Привет', index: 0, input: 'Привет, мир!', isWordLike: true }
// { segment: ',', index: 6, input: 'Привет, мир!', isWordLike: false }
// { segment: ' ', index: 7, input: 'Привет, мир!', isWordLike: false }
// { segment: 'мир', index: 8, input: 'Привет, мир!', isWordLike: true }
// { segment: '!', index: 11, input: 'Привет, мир!', isWordLike: false }
console.log(...wordSplitter.segment("Привет, мир!"));

// 3. Сегментация на предложения
const sentenceSplitter = new Intl.Segmenter("ru", { granularity: "sentence" });

// { segment: 'Привет! ', index: 0, input: 'Привет! Как дела?' }
// { segment: 'Как дела?', index: 8, input: 'Привет! Как дела?' }
console.log(...sentenceSplitter.segment("Привет! Как дела?"));
```


РЕФЛЕКСИРУЕМ

- * Стандартный итератор строки **обходит строку по кодовым точкам** Юникода
- * API [Intl.Segmenter](#) позволяет создать итератор, который обходит строку тремя разными способами (гранулярность)
- * **По графемам** (grapheme) – пользовательские символы
- * **По словам** (word) – с учётом языка и разделителей
- * **По предложениям** (sentence) – с учётом пунктуации

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

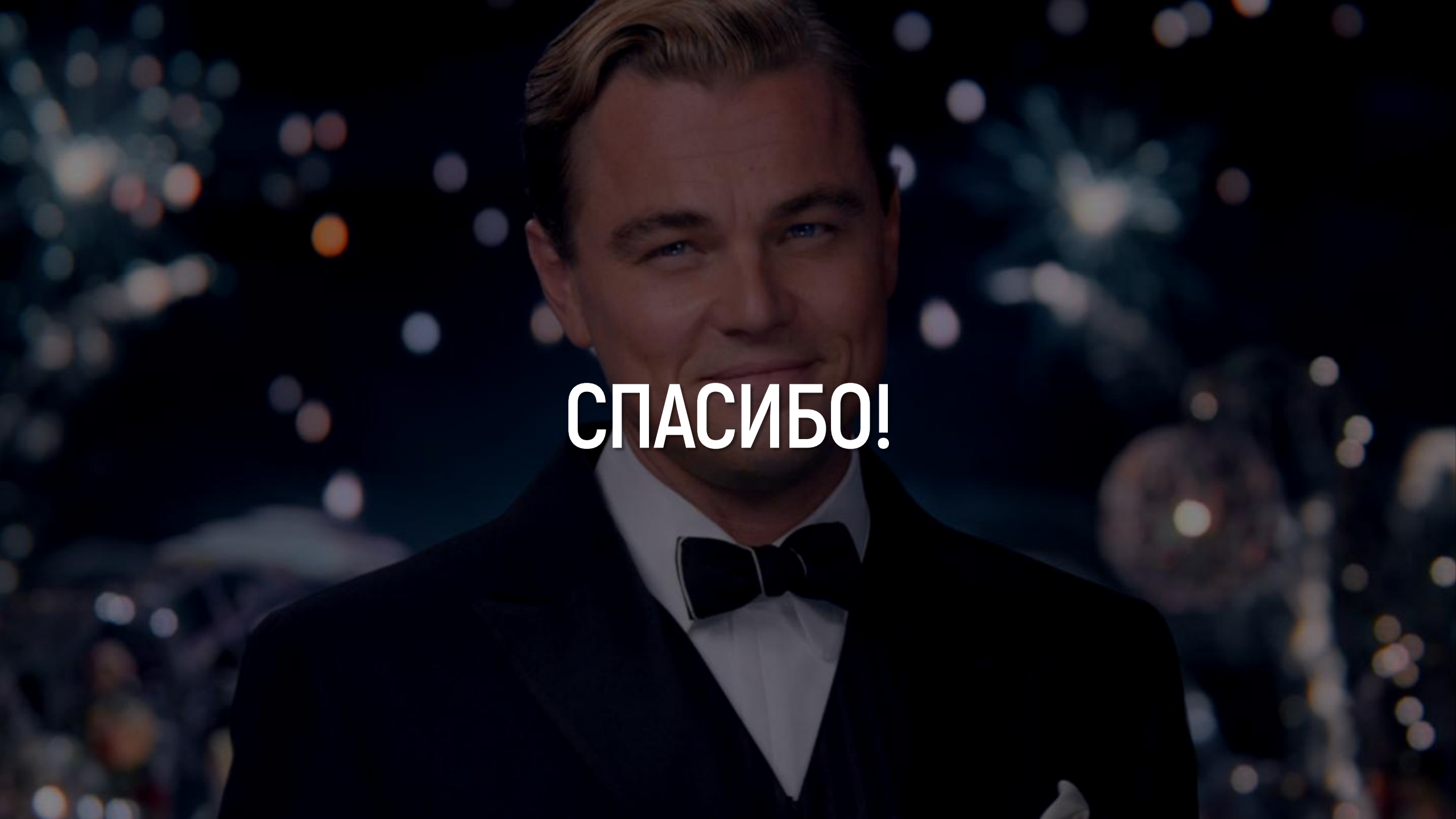
СТОЖКА, СТОЖКААА!

ЮНИКОД – ЭТО СЛОЖНО

- * Очень много нюансов
- * Как правило, информация запоминается ценой часов и дней дебага
- * Но я в вас верю 😊

ПОДВЕДЁМ ИТОГИ

- * Юникод - это набор символов, где у каждого символа есть **свой код (21 бит, обычно хранится в 32-битном числе)**
- * Символы Юникода **должны кодироваться** с использованием специальных кодировок
- * В JavaScript **используется кодировка UTF-16**
- * В UTF-16 символы Юникода кодируются либо одним, **либо двумя 16-битными числами**
- * Некоторые символы Юникода **могут изменять отображение других символов**
- * Помните про встроенные API

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus bokeh lights in various colors (white, yellow, orange, blue), suggesting a night-time event or fireworks.

СПАСИБО!